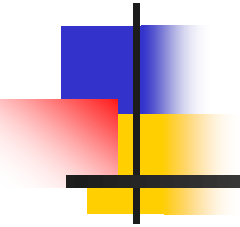
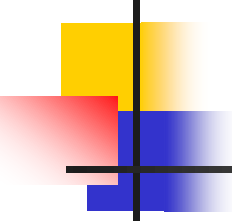


# 10. Generics

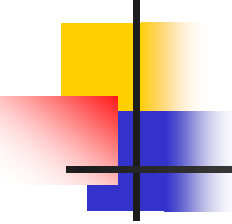




# Generics

---

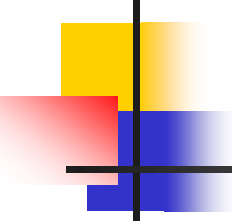
- Generics in Java allow you to define classes, interfaces, and methods with type parameters. They provide a way to create reusable code components that can handle any type of object while maintaining type safety. With generics, you can write code that works with different types of data without sacrificing compile-time type checking.



# Generics

---

- The type is represented by one or more type parameters (**like T, E, K, V**), which are placeholders for actual data types that are provided when the class is instantiated.
- A generic class provides flexibility and reusability, as it can handle different types without needing to write separate versions of the class for each type.

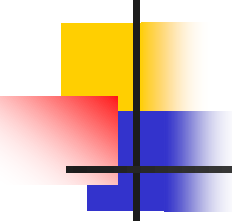


# Generics

---

- Objective of a Generic Class

1. **Type Safety:** Generic classes allow you to catch type errors at compile-time instead of runtime. This ensures that the code is safer and more predictable.
2. **Reusability:** By defining a generic class, you can reuse it with different types of objects without duplicating code.



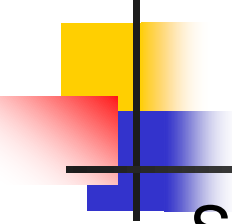
# Generics

---

- Objective of a Generic Class

3. **Code Clarity:** Generics make the code easier to understand by providing a clear distinction of the type being used.

4. **Elimination of Type Casting:** With generics, you don't have to cast objects to their proper types, which reduces errors and makes the code cleaner.



# Generics

---

- Syntax of a Generic Class

The general syntax of a generic class looks like this:

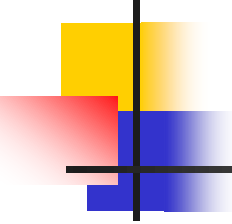
```
class ClassName<T> {
```

```
// Fields, methods, and constructors can use 'T' as the type
```

```
}
```

**T** is the type parameter, which will be replaced with an actual type when the class is used.

You can use multiple type parameters like **<T, U, V>** to handle more than one type.

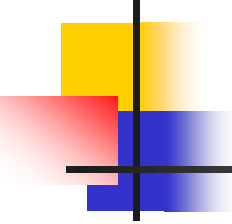


# Generics

---

- **Key Concepts**

1. Type Parameters:
2. Generic Classes:
3. Generic Methods:
4. Bounded Type Parameters:
5. Wildcards (?):

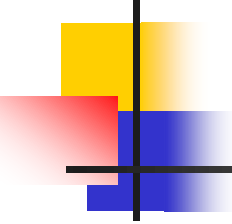


# Generics

---

- **Type Parameters**

When you define a generic class, interface, or method, you can specify one or more type parameters. These are placeholders for actual types that are provided when the code is used.

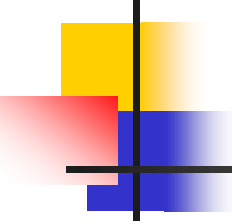


# Generics

---

```
public class Box<T> {  
    private T value;  
  
    public void set(T value) {  
        this.value = value;  
    }  
  
    public T get() {  
        return value;  
    }  
}
```

Here, T is a type parameter, and the actual type is specified when you create an object of the class, like **Box<Integer>** or **Box<String>**.



# Generics

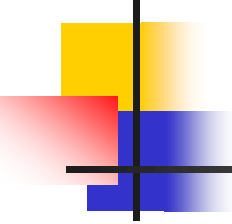
---

- **Generic Classes**

A class that can work with any data type. The type is specified when you instantiate the class.

```
Box<String> stringBox = new Box<>();
```

```
Box<Integer> integerBox = new Box<>();
```



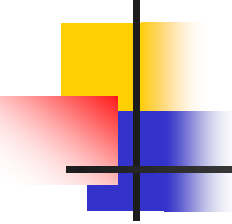
# Generics

---

- **Generic Methods**

A method that can operate on objects of various types, specified when the method is called.

```
public static <T> void printArray(T[] array) {  
    for (T element : array) {  
        System.out.println(element);  
    }  
}
```



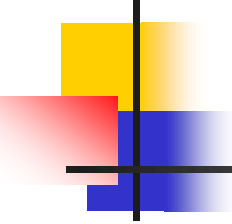
# Generics

---

- **Bounded Type Parameters:**

You can restrict the types that can be used as arguments by using `extends` or `super`.

```
public <T extends Number> void printNumber(T num) {  
    System.out.println(num);  
}
```



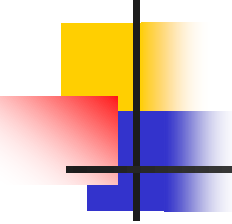
# Generics

---

- **Bounded Type Parameters:**

```
public <T extends Number> void printNumber(T num) {  
    System.out.println(num);  
}
```

This ensures that only `Number` or its subclasses (like `Integer`, `Double`, etc.) can be passed as the type parameter.



# Generics

---

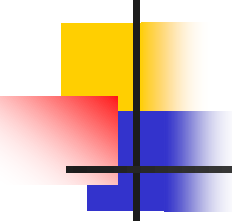
- **Wildcards (?):**

- Sometimes, you may not know the exact type, and you want to represent any type. You use wildcards to achieve this.

- **Unbounded Wildcard:**

- `<?>` can accept any type.

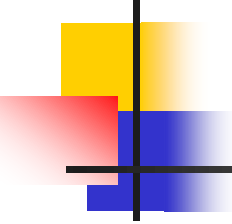
```
List<?> list = new ArrayList<String>();
```



# Generics

---

- **Wildcards (?):**
  - **Upper Bounded Wildcard:**
    - `<? extends T>` limits the type to T or its subclasses.  
`List<? extends Number> numbers = new ArrayList<Integer>();`
  - **Lower Bounded Wildcard:**
    - `<? super T>` limits the type to T or its superclasses..  
`List<? super Integer> integers = new ArrayList<Number>();`



# Generics

---

| Wildcard                   | Usage                                       | Example                             |
|----------------------------|---------------------------------------------|-------------------------------------|
| <b>&lt;?&gt;</b>           | <b>Any type</b>                             | <b>List&lt;?&gt;</b>                |
| <b>&lt;? extends T&gt;</b> | <b>Specific type T and its subclasses</b>   | <b>List&lt;? extends Number&gt;</b> |
| <b>&lt;? super T&gt;</b>   | <b>Specific type T and its superclasses</b> | <b>List&lt;? super Integer&gt;</b>  |